Responsive Design y accesibilidad
Buenas y malas prácticas. Errores comunes

Autor: Olga Carreras

Artículo permanente:

Responsive Design y accesibilidad. Buenas y malas prácticas. Errores comunes.

Índice

Responsive Design. ¿Qué es? 3

Responsive Design. ¿Cómo se hace? 5

Relación entre Responsive Design y accesibilidad 9

Concretando, ¿cómo puede favorecer a la accesibilidad de un sitio que este sea Responsive Design? 10

No es oro todo lo que reluce 11

Primera norma de accesibilidad para Responsive Design 12

Cinco ejemplos de errores de accesibilidad consecuencia de una mala ocultación de contenidos 13

Otras consideraciones de accesibilidad a tener en cuenta 22

Malas prácticas 22

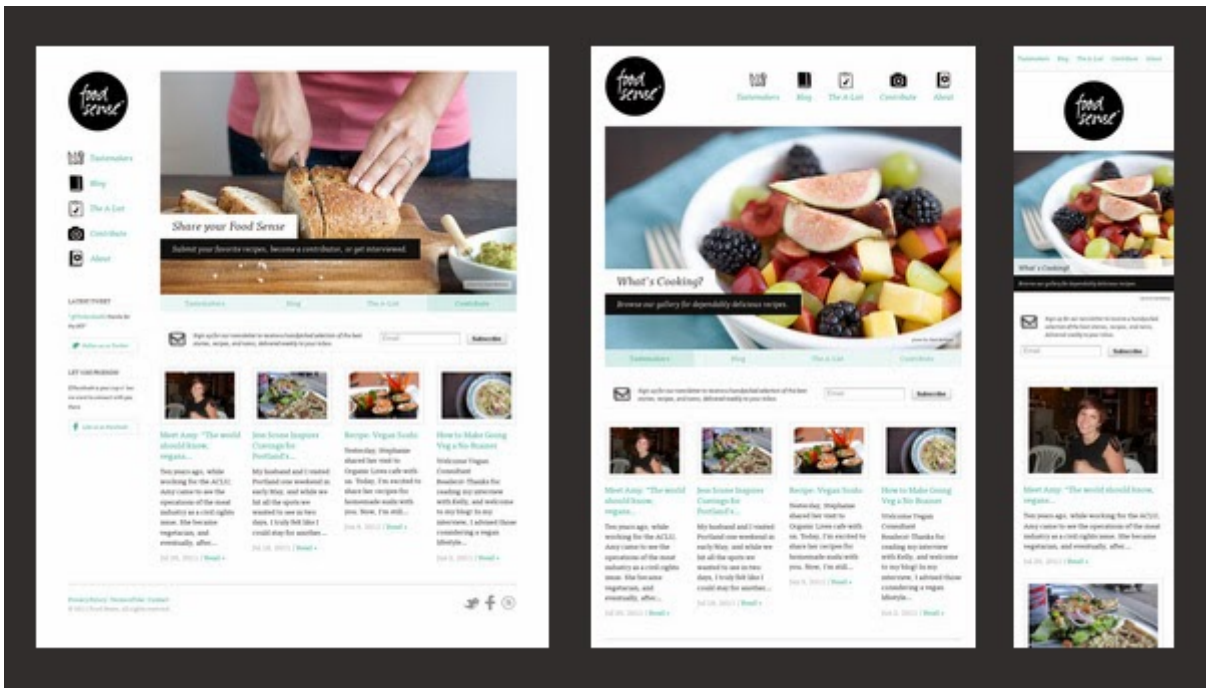
Buenas prácticas 25

Bibliografía 29

Artículos relacionados 30

Responsive Design. ¿Qué es?

Responsive Web Design (RWD) es una técnica de diseño y desarrollo de sitios y aplicaciones web que permite que las páginas se adapten al tamaño, la resolución y orientación de la pantalla, y por tanto al dispositivo del usuario. Y todo ello con un código único, una única página, una única URL.



http://foodsense.is/

Tenéis 50 ejemplos de Responsive Web Design en el artículo "Responsive Web Design: 50 Examples and Best Practices", designmodo.com, octubre 2011

No necesitas realmente acceder a estas páginas en diferentes dispositivos para comprobar que son Responsive Web Design, basta con que las visualices desde tu escritorio y redimensiones la pantalla del navegador. Según el tamaño de la pantalla verás que el diseño, la navegación, el contenido y las imágenes se reconfiguran automáticamente.

También puedes cambiar la resolución de pantalla, selecciona una resolución baja y podrás ver la visualización de la web para esa resolución.

A medida que hacemos la pantalla más pequeña o bajamos la resolución, pasamos por ejemplo de un *layout* de 3 columnas a uno de 2 y finalmente a un *layout* de una columna. Las imágenes se hacen más pequeñas para adaptarse al espacio disponible o el sistema de navegación se modifica.

http://mashable.com/

Podéis consultar el artículo de Juan Carlos Mejía [MEJIA, 2012] donde incluye varios vídeos que ilustran todo ello con claridad.

Responsive Design. ¿Cómo se hace?

Esta flexibilidad se consigue mediante el uso de un código HTML único pero que se presenta de manera diferente gracias a:

- **La separación entre el contenido y la presentación:** todos los estilos están definidos en las CSS.
- ***Layouts* basados en *grids*:** la información se organiza en ejes verticales y horizontales. Tendremos definidos diferentes *layouts* en diferentes CSS, por ejemplo uno de 3 columnas, uno de 2 columnas y uno de 1 columna.

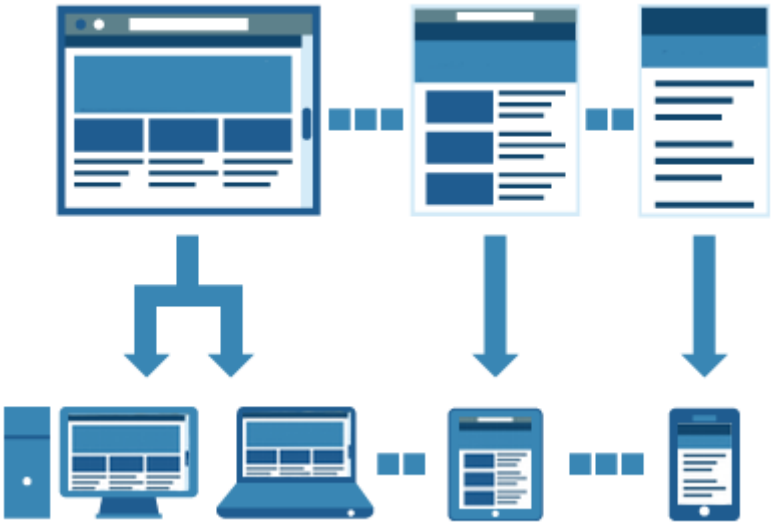


Imagen de boatboatool.com

- **Fluids Grids**, es decir, el uso de medidas relativas que permitan que el contenido se pueda adaptar realmente, como se ve en la imagen anterior. Permite utilizar todo el espacio disponible y evitar el desplazamiento horizontal.
- **Media Queries**, permite cargar dinámicamente las diferentes CSS que hemos definido en función del tamaño de pantalla, su resolución o su orientación.

```
<link rel="stylesheet" type="text/css" href="style2col.css"
media="all and (min-width: 400px) and (max-width: 800px)" />
```

En el ejemplo de código anterior se especifica la CSS a utilizar (*layout* de 2 columnas) con un *viewport* (la parte de pantalla donde se representa el documento) de una anchura entre 400px y 800px.

Media Queries son recomendación del W3C desde junio de 2012.

En la bibliografía, al final del post, tienes algunos artículos relacionados con la resolución y tamaño de los diferentes dispositivos y cómo aplicar Media Queries: [CHELARIU, 2013], [CARRERAS, 2014], [ANDROID], [PRIETO, 2014]

- **Configuración del meta *viewport***: Mediante este *meta* indicamos que nuestra web es flexible para adaptarse a los diferentes anchos y resoluciones de pantalla, le indicamos al navegador que no aumente o reduzca la página, que use el zoom por defecto.

```
<meta name="viewport" content="width=device-width,initial-
scale=1.0" />
```

Con el uso combinado de Media Queries y la definición del *viewport* evitamos que nuestra web se vea en miniatura en un dispositivo móvil, que tengamos que hacer zoom y después tener que desplazarnos porque el contenido ocupa más que el ancho de pantalla. Por el contrario, ahora se verá a tamaño real y ajustado a todo el ancho.

Sin embargo más adelante veremos que es importante, desde un punto de vista de accesibilidad, no impedir que el usuario pueda hacer zoom si lo desea (ver mala práctica 2). Indicaré cómo evitar una incorrecta configuración del *viewport*.

- **Imágenes y vídeos de tamaño flexible**, que también se adapten al espacio disponible. Se puede conseguir de diferentes maneras, cada una con sus ventajas e inconvenientes. Más adelante hablaré de cómo algunas de ellas tienen implicaciones en la accesibilidad de la página.

En el caso de los vídeos podemos usar HTML5 y el elemento VIDEO con diferentes *sources* según la resolución. Tenéis un ejemplo en el artículo de Ian Devlin [DEVLIN, 2012]

En el caso de las imágenes no es tan sencillo, por ello hay voces pidiendo un atributo *srcset* para el elemento IMG o un elemento PICTURE con diferentes *sources* como el elemento VIDEO [W3C_GROUP(b), 2012]

A falta de estas opciones, se suelen utilizar las siguientes técnicas:

- **La imagen se define como fondo de un elemento en las CSS (*background-image*)** Se tienen diferentes versiones de la imagen a diferentes tamaños, en cada CSS se carga una de ellas. Veremos que aunque es más óptimo para aligerar el peso de las páginas, puede suponer un problema grave de accesibilidad si las imágenes así definidas no son decorativas sino informativas.

- **La imagen se incluye en el código HTML con el elemento IMG pero se define su anchura y altura en las diferentes CSS**. El gran inconveniente es que aunque muestras la imagen a diferente tamaño en realidad se carga siempre la de mayor peso.

- **Los logotipos e iconos se incluyen con el formato .svg** Podéis consultar varios ejemplos en [PUKHALSKI, 2014] [SOUEIDAN, 2014]

- **Otras opciones que ya no son propiamente Responsive Design** son detectar el dispositivo en el servidor para servir las diferentes versiones de la imagen según el dispositivo, o usar javascript, cookies [W3C_GROUP, 2012].

En cuanto a la metodología de trabajo es muy importante **tener ya presente**, no solo en la etapa de diseño, sino también **en la de conceptualización y prototipado**, cómo será el sitio en función del tamaño de pantalla.

Desde 2009 Luke Wroblewski viene defendiendo un enfoque **Mobile First**, basado en el principio de Mejora Progresiva. Pensar primero en la versión móvil y después ir añadiendo complejidad mediante Media Queries, que serán ignoradas por los navegadores que no las soporten.

Este enfoque, dada las limitaciones de las pantallas pequeñas, **te permite centrarte en las necesidades reales de los usuarios, priorizar las tareas claves:**

Los dispositivos móviles requieren que los equipos de desarrollo de software se centren únicamente en los contenidos y en las acciones más importantes de la aplicación. Simplemente no hay espacio en una pantalla de 320x480px para los elementos innecesarios. Hay que priorizar.

Así que, cuando un equipo diseña primero para el móvil, el resultado final es una experiencia centrada en las tareas clave que los usuarios quieren lograr, sin distracciones […] Eso es bueno para la experiencia de usuario y bueno para los negocios.

[WROBLEWSKI, 2009]

Para ampliar cómo se aplica técnicamente Mobile First, cómo se define primero la CSS para los tamaños de pantalla más pequeños impidiendo que esta sea la que se cargue en versiones anteriores a IE9, se puede leer el artículo de Ricardo Prieto [PRIETO, 2014].

Relación entre Responsive Design y accesibilidad

Un sitio desarrollado con la técnica Responsive Design no implica que dicho sitio vaya a ser accesible y a cumplir con las pautas de accesibilidad [WCAG2, 2008], sin embargo es un gran punto de partida.

Parten de un enfoque o filosofía similar: defienden una web única, que los sitios sean flexibles, independientes del dispositivo y a disposición de todos los usuarios.

Hay ciertos **requisitos de accesibilidad**, a nivel de código, que tienen gran impacto en la accesibilidad de las páginas y que deben tenerse en cuenta desde el comienzo del desarrollo, pues son muy costosos de corregir a posteriori: el uso de estándares, la separación entre el contenido y la presentación, el uso de medidas relativas, evitar las tablas para maquetar o la definición de jerarquías de información estructuradas correctamente.

Como hemos visto, **un sitio Responsive Design cumple per se con muchos de estos requisitos.**

Por otra parte, también hemos comentado que la **metodología Mobile First** va más allá de la mera adaptación del sitio al tamaño de pantalla. Ahora, cada pieza de información, cada enlace, debe ganarse su lugar.

Esto ayuda a **priorizar contenidos y funcionalidades eliminado lo innecesario:** las páginas son más cortas y sencillas y la navegación más racional. Como resultado tendremos sitios más fáciles de navegar y entender, con menor carga cognitiva y visual.

Como defiende Luke Wroblewski [WROBLEWSKI, 2010] la metodología Mobile First para **Responsive Design ayuda a que las páginas sean más accesibles**. Favorece a todos los usuarios, pero sin duda especialmente a los usuarios con discapacidad cognitiva, a los usuarios que utilizan lectores de pantalla o a los usuarios que tienen una discapacidad motora y utilizan dispositivos de entrada alternativos.

Concretando, ¿cómo puede favorecer a la accesibilidad de un sitio que este sea Responsive Design?

- **El contenido y la presentación están separados**, los estilos están definidos en las CSS y no se usan tablas para maquetar. Todo ello beneficia a las personas con diferentes discapacidades al permitir a los agentes de usuario adaptar el contenido de acuerdo a sus necesidades (criterio de conformidad 1.3.1, [WCAG2, 2008])

- **Tendencia a un mayor respeto por los estándares web**, lo cual maximizará la compatibilidad con las aplicaciones de usuario actuales y futuras, incluyendo las ayudas técnicas (pauta 4.1, [WCAG2, 2008])

- **Tendencia a tener la información estructurada y jerarquizada más correctamente** (criterio de conformidad 1.3.1, [WCAG2, 2008])

- **Tendencia al uso de elementos semánticos** para poder definir sus estilos en las CSS. Indicar explícitamente la función estructural o valor semántico del contenido permitirá que esta información se pueda determinar mediante software favoreciendo la accesibilidad (criterio de conformidad 1.3.1, [WCAG2, 2008])

- El diseño flexible y la definición de tamaños relativos **permite que el texto se pueda ampliar sin desbordamientos y hacer zoom con garantías** (criterio de conformidad 1.4.4, [WCAG2, 2008])

    - El tamaño flexible de las imágenes y vídeos permitirá que se adapten mejor al espacio disponible sin que se superpongan con otros contenidos.

    - Mejorará la experiencia de los usuarios con baja visión que suelen tener resoluciones de pantalla más bajas y suelen ampliar la pantalla.

- Además, el diseño flexible y que no se usen tablas para maquetar **ayuda a garantizar un orden de lectura correcto**. Los diseños fluidos tienden a presentar el contenido en el mismo orden que el DOM y este es el mismo orden en que, por ejemplo, los lectores de pantalla leerán el contenido (criterio de conformidad 1.3.2, [WCAG2, 2008])

- **Se tiene muy presente que el sitio se visualizará en distintos dispositivos** y por tanto:

    - Es más probable que tu sitio no sea operable solo con el ratón. La forma de interactuar ahora es muy variable y esto ayuda al diseño inclusivo (principio Operable, [WCAG2, 2008])
    - Es más probable que mejores el contraste de color, que suele ser más pobre en los dispositivos móviles, ya que bajamos el brillo para ahorrar batería, además de que son habituales los reflejos en la pantalla (criterios de conformidad 1.4.3 y 1.4.6, [WCAG2, 2008])
    - Mayor uso de la técnica **Progressive Enhancement (Mejora Progresiva)** que consiste en una implementación básica, que funciona a través de múltiples dispositivos y con una amplia gama de tecnologías de asistencia, añadiendo después más funcionalidades para los dispositivos que las soportan.
- **Focalizarse solo en lo necesario, priorizar y simplificar**, dará como resultado sitios más fáciles de navegar y entender, con menor carga cognitiva y visual, mejorando la legibilidad y la accesibilidad.
    - Mayor uso de la técnica **Progressive Disclosure (Revelación Progresiva)**, que se basa en diferenciar el contenido primario del secundario. El contenido primario aparece inmediatamente en el flujo normal de la página y es muy visible. El objetivo es mostrar solo lo relevante para el usuario en este momento. Nos beneficia a todos, pero especialmente a los usuarios con discapacidad cognitiva o con déficit de atención, y bien hecho facilita la navegación a los usuarios de lectores de pantalla o a las personas con discapacidad motora.

No es oro todo lo que reluce

Si terminara el artículo aquí podría parecer que Responsive Design es la panacea para la accesibilidad, pero no es así. La mayoría de las veces los desarrollos no cumplen con todos los puntos enumerados anteriormente. También nos encontramos con problemas recurrentes y malas prácticas que suponen barreras de accesibilidad en los desarrollos Responsive Design.

A continuación el artículo se centra en dichos problemas.

Primera norma de accesibilidad para Responsive Design

Como hemos visto, la accesibilidad no tiene por qué verse comprometida en los sitios Responsive Design, sino que por lo general suelen cumplir con ciertos requisitos de accesibilidad y es una buena base para comenzar a trabajarla.

Sin embargo no siempre es así, habrá ciertos aspectos a los que prestar especial atención porque **nos encontramos con problemas recurrentes y extendidas malas prácticas.**

> **La primera norma**, y más importante, es:
>
> hay que comprobar la accesibilidad en las diferentes resoluciones establecidas mediante Media Query.

Puede parecer una tontería, puesto que hemos dicho que la página es la misma y que simplemente se cargan distintas CSS, pero no lo es.

Muchas veces se ocultan contenidos en las versiones para tamaños de pantalla más pequeños. Para que determinado contenido no se muestre, a veces lo ocultan con estilos definidos en la CSS para las resoluciones más bajas. Otras veces se hace detectando el dispositivo desde el servidor o por javascript. Todo eso no es Responsive Design, pero son prácticas habituales que, mal hechas, suelen generar problemas de accesibilidad.

A continuación voy a poner **5 típicos ejemplos de errores de accesibilidad** que:

- NO encontramos cuando accedemos a la página con el mayor tamaño o resolución definido,
- pero SÍ encontramos cuando reducimos la resolución o el tamaño de pantalla, y **que son consecuencia de la ocultación de contenido mal implementado o sin medir sus consecuencias**.

Cinco ejemplos de errores de accesibilidad consecuencia de una mala ocultación de contenidos

1. Los encabezados ya no tienen una jerarquía correcta

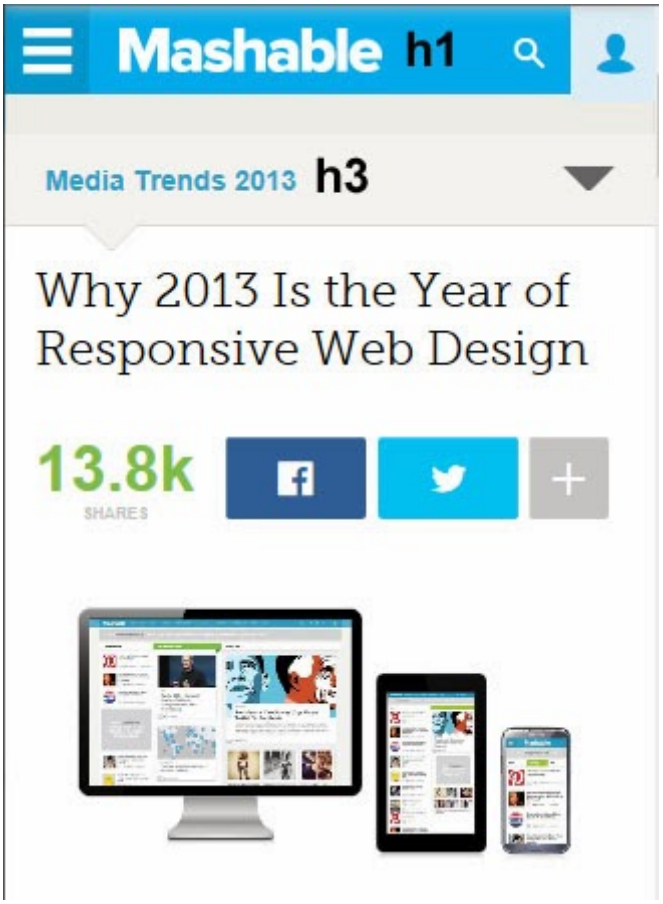
Hemos dicho que no solo se adapta la visualización sino también a menudo el contenido, eliminando zonas de información en las versiones con resoluciones menores. Cuando cierto contenido desaparece la jerarquía de encabezados puede verse comprometida.

Veamos la página Mashable.

Tenemos un encabezado H1 con el logo, un H2 con la categoría “Tech” y un H3 “Media Trends 2013” (en realidad en la página se saltan un nivel y “Media Trends 2013” es H4, pero vamos a imaginarnos que lo hubieran hecho bien porque para el ejemplo es indiferente)

Encabezados de Mashable a una resolución alta

En la imagen que sigue, vemos la misma página al tamaño más reducido de pantalla.



Encabezados de Mashable a una resolución baja

El contenido se ha simplificado.

Se ha eliminado el H2 “Tech” (tanto visualmente como para los usuarios de lectores de pantalla) junto con la barra de redes sociales.

Como consecuencia, tras el H1, nos encontramos con un H3. **La jerarquía de encabezados ya no es consistente y esto dificulta ojear el documento cuando accedes con el lector de pantalla**.

2. Enlaces para saltar el contenido que ahora solo crean ruido

Estamos en un caso similar al anterior.

Si has eliminado o simplificado contenido debes comprobar que los enlaces para saltar contenido ahora siguen teniendo sentido, o si simplificada la página, estos solo crean ruido.

Lo mismo puede ocurrir con otros elementos, como los WAI-ARIA landmarks (ver artículo Navegación más accesible y semántica en 2 minutos con Landmark Roles (WAI-ARIA)).

Hay un artículo de Henney Swan [SWAN (b), 2012] que habla sobre los enlaces de saltar contenido en Responsive Design.

3. El contenido se oculta de forma inadecuada

En el caso de que estemos ocultando contenido en las CSS es importante comprobar cómo se está ocultando.

En el artículo "Ocultar contenido sin comprometer la accesibilidad ni el posicionamiento de la página" explicaba las diferentes técnicas para ocultar contenido.

En función de cómo lo ocultes, el contenido estará también oculto o no para los lectores de pantalla:

- **Display:none o visibility:hidden**, el contenido tampoco estará disponible para los lectores de pantalla.
- **Text-indent:-9999 o la ocultación mediante clip**, el contenido sí estará disponible para los lectores de pantalla.

Si en la versión para tamaños de pantalla pequeños has ocultado contenido al usuario para simplificar la página, asegúrate de que también esté oculto para los lectores de pantalla, de esta manera estarán en igualdad de condiciones.

Si ocultas unos contenidos con una técnica y otros con otra, sin ningún criterio, la página puede llegar a ser incomprensible para ellos.

En el apartado 4, sobre problemas en la navegación, veremos también un ejemplo de contenido oculto de forma incorrecta.

4. El menú de navegación ahora es diferente, ¿es accesible?

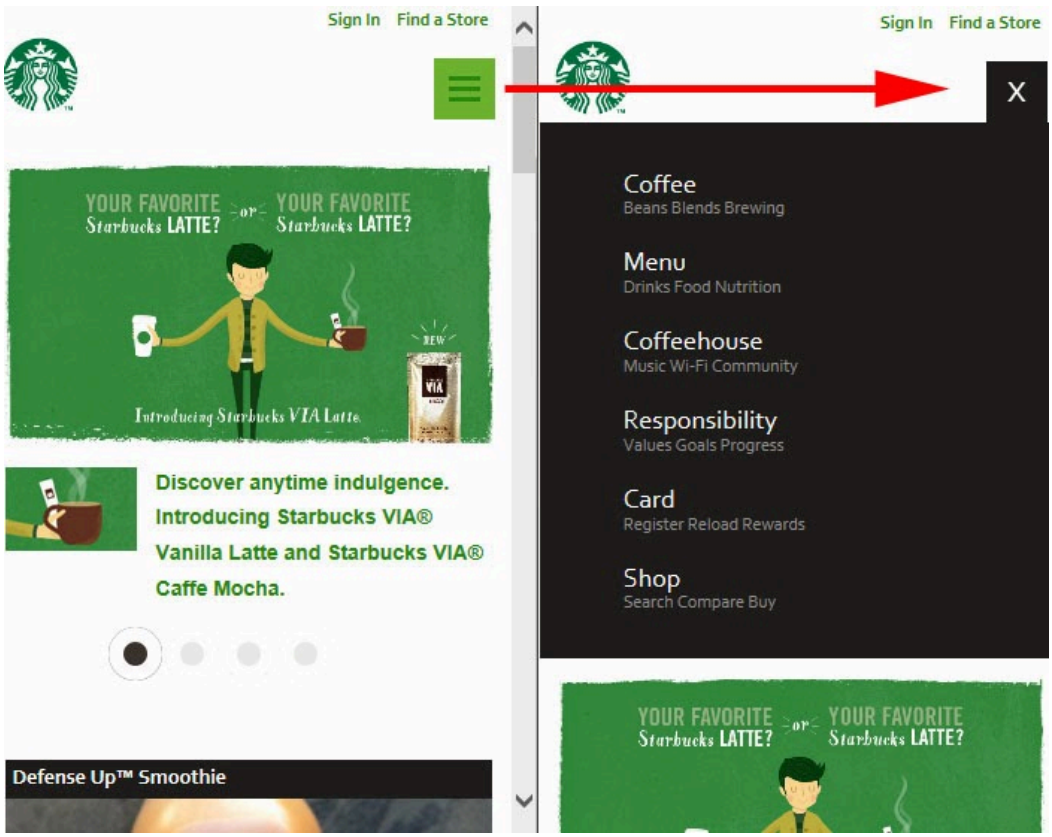
En las versiones para las menores resoluciones o tamaños de pantalla el menú suele convertirse hoy en día, casi un estándar de facto, en un icono con tres rayitas que se despliega.

Pasamos de tener un menú lineal y visible a un menú desplegable, y es por tanto importante asegurarse de que la navegación sigue siendo accesible para todos los usuarios.

Los **tres ejemplos** que pongo a continuación están basados en la modificación de las CSS, **el código HTML es el mismo para las dos visualizaciones**.

Ejemplo 1: http://www.starbucks.com/

Menú de navegación con resoluciones de pantallas mayores



Menú de navegación con resoluciones de pantallas menores

En la segunda versión, la versión para resoluciones de pantalla más pequeñas, el lector de pantalla nos anuncia el icono “Enlace Navigation” ( ), si lo pulso y continuo, me anuncia “Lista con seis elementos” y podemos acceder a sus enlaces. Sin problemas.

Pero si después de que me anuncie “Enlace ‘Navigation’” yo decido seguir adelante sin pulsarlo porque no me interesa acceder a la navegación… vaya, también me anuncia “Lista con seis elementos”. Me encuentro con la navegación aunque ni la he solicitado ni está visible. Resulta confuso y molesto. No la han ocultado correctamente.

Ejemplo 2: http://antocas.com/demos/menu-responsive-design/

Menú de navegación con resoluciones de pantallas mayores

Menú de navegación con resoluciones de pantallas menores

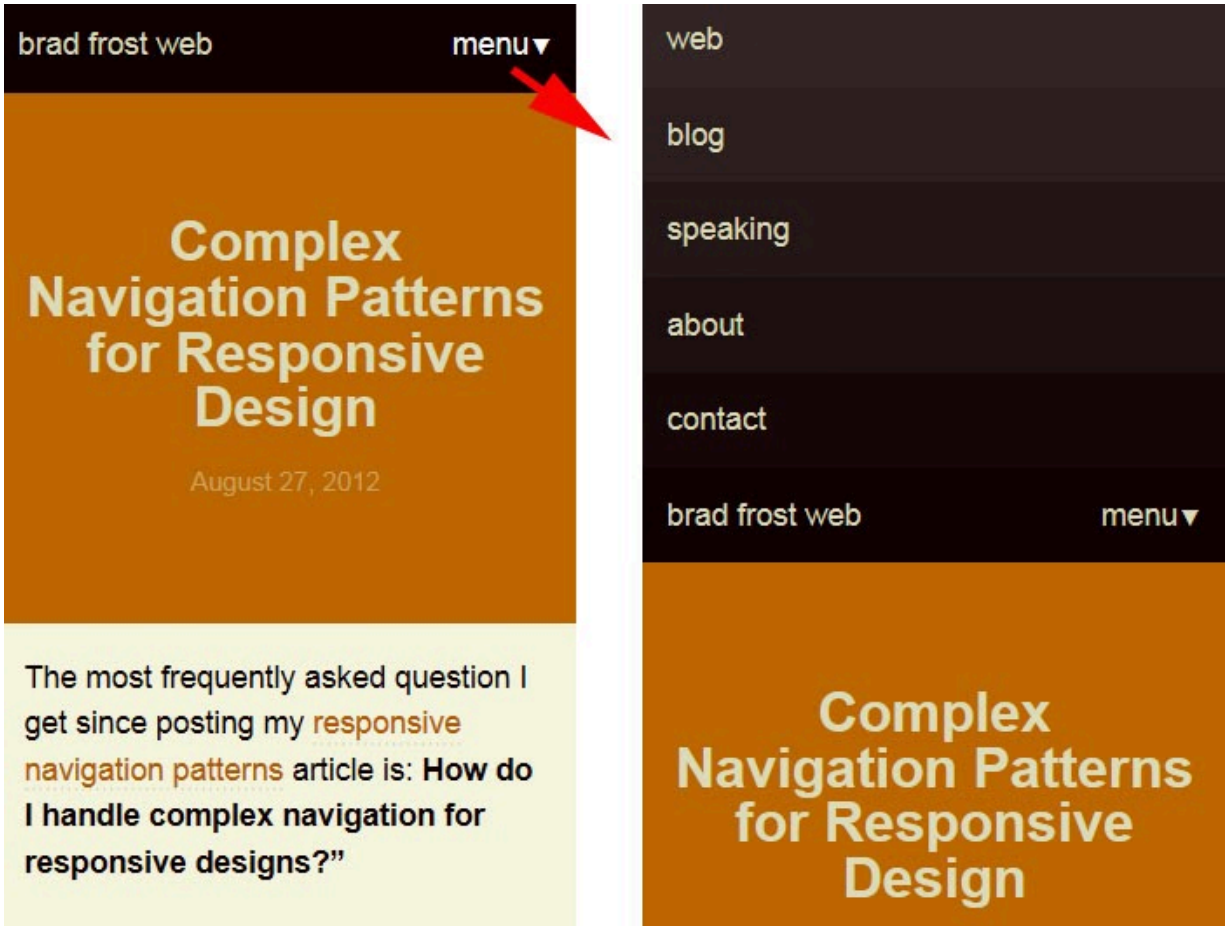
En la segunda versión, la versión para resoluciones de pantalla más pequeñas, el lector de pantalla me anuncia el icono “Enlace ‘Nav Menu’” ( )

Si no lo pulso accedo al resto del contenido de la página correctamente, al contrario que en el ejemplo anterior.

Sin embargo, cuando lo pulso… no consigo encontrar el supuesto menú desplegado. ¿Por qué? Porque **el orden de lectura no es correcto. El código del menú está encima del código del icono**. Tengo que adivinar que solo pulsando por ejemplo la flecha arriba conseguiré llegar a él.

Ejemplo 3: http://bradfrostweb.com/blog/web/complex-navigation-patterns-for-responsive-design/

Menú de navegación con resoluciones de pantallas mayores



Menú de navegación con resoluciones de pantallas menores

En la segunda versión, la versión para resoluciones de pantalla más pequeñas, **el usuario de lector de pantalla nunca puede acceder al enlace "Menú"**.

Lee siempre las opciones de menú al comienzo de la página como si estuviera en la navegación para escritorio, como si siempre estuvieran visibles, esté o no el menú desplegado. No se ha manejado correctamente la ocultación de los elementos.

5. Asegúrate de que el orden de lectura ahora también es correcto.

Por último, un desarrollo *responsive* suele favorecer que el orden de lectura sea adecuado, pero cuando se empieza a ocultar contenido es importante asegurarse de que el orden de lectura sigue siendo correcto.

En el punto anterior ponía un ejemplo (ejemplo 2) de un orden de lectura incorrecto en un menú de navegación desplegable.

Otras consideraciones de accesibilidad a tener en cuenta

Hemos visto en el apartado anterior errores comunes relacionados con ocultar contenido en las versiones móviles de manera inadecuada o sin tener en cuenta las consecuencias para la accesibilidad de la página.

Ahora vamos a ver otras malas y buenas prácticas que hay que tener en cuenta.

Malas prácticas

Explico a continuación tres malas prácticas que hay que evitar.

Mala práctica 1: Tratar imágenes informativas como imágenes decorativas para cargar diferentes tamaños de imagen

Si incluyes las imágenes en el código y quieres mediante las CSS que se muestren a diferente tamaño, puedes definir su ancho y su alto en las diferentes CSS. El problema es que la imagen que descargas es siempre la misma, la más grande, y solo estas cambiando su tamaño de visualización.

Para intentar evitarlo, **un error común** es tratarlas como imágenes decorativas, de fondo (definidas en el *background-image* de un elemento) de esta manera puedes tener diferentes tamaños de la imagen y en cada CSS cargar una.

Utiliza cualquier otra técnica menos esta: **tratar imágenes informativas como imágenes decorativas supone un grave problema de accesibilidad**, pues las imágenes decorativas no tienen una alternativa textual y si, por diferentes motivos, no se cargan o el usuario no puede verlas (por ejemplo accede con un lector de pantalla), se perderá la información que transmiten.

Mala práctica 2: Definir el viewport con restricción para el zoom

Si el usuario lo desea debe poder hacer zoom con el gesto “pinch” ( ) al menos un 200% (criterio de conformidad 1.4.4, [WCAG2, 2008]). Para ello es importante definir correctamente el *viewport*:

Mala práctica:

```
<meta name="viewport" content="width=device-width;initial-scale=1.0;
maximum-scale:1.0; user-scalable=1" />
```

Defiendo así el *viewport* estamos impidiendo el zoom. Podemos ver un ejemplo en la web http://www.anderssonwise.com/

Buena práctica

```
<meta name="viewport" content="width=device-width;initial-scale=1.0;
maximum-scale:2.0; user-scalable=1" /<
```

Definiendo así el *viewport* estamos permitiendo que los usuarios que lo deseen puedan hacer un zoom de hasta 200%.

Mala práctica 3: Ocultar la barra de scroll horizontal

No debes ocultar la barra de scroll horizontal con *overflow-x: hidden;*

Tu sitio ya se adapta al dispositivo sin barras de scroll, ¿para qué la ocultas además si no es necesario? Puede haber circunstancias en las que los usuarios la necesiten, por ejemplo si hacen zoom.

Buenas prácticas

A continuación indico cuatro buenas prácticas que deberías llevar a cabo en tu desarrollo Resposive Design. Y por supuesto la mejor recomendación es que las páginas cumplan con las pautas de accesibilidad WCAG 2.0

Buena práctica 1: Contraste de color mejorado en la versión para las resoluciones o tamaños de pantalla más pequeños.

Para alcanzar el nivel de adecuación AA respecto a las WCAG 2.0, el color de los textos debe tener al menos un ratio de contraste de 4.5:1 (3:1 en texto grandes) Para alcanzar el nivel AAA el contraste debe ser de 7:1 (4.5:1 en textos grandes)

Puesto que tenemos una CSS para dispositivos móviles os animo a que en ella alcancéis un nivel de contraste de color mayor, llegando al nivel AAA. En los dispositivos móviles el contraste suele ser menor, para ahorrar batería, y solemos tener muchos reflejos. También es importante que subrayéis los enlaces para reconocerlos mejor.

Se puede comprobar el contrate de color con la herramienta gratuita Colour Contrast Analyser.

Buena práctica 2: Diseña el tamaño de los elementos y la separación entre los mismos pensando también en los dispositivos móviles

En la web para desarrolladores de Android [ANDORID] tenéis resumida de forma gráfica cuáles deberían ser los tamaños y espacios mínimos para que tus elementos sean fáciles de pulsar desde dispositivos táctiles. Nos ayuda a todos y especialmente a las personas con discapacidad motora:

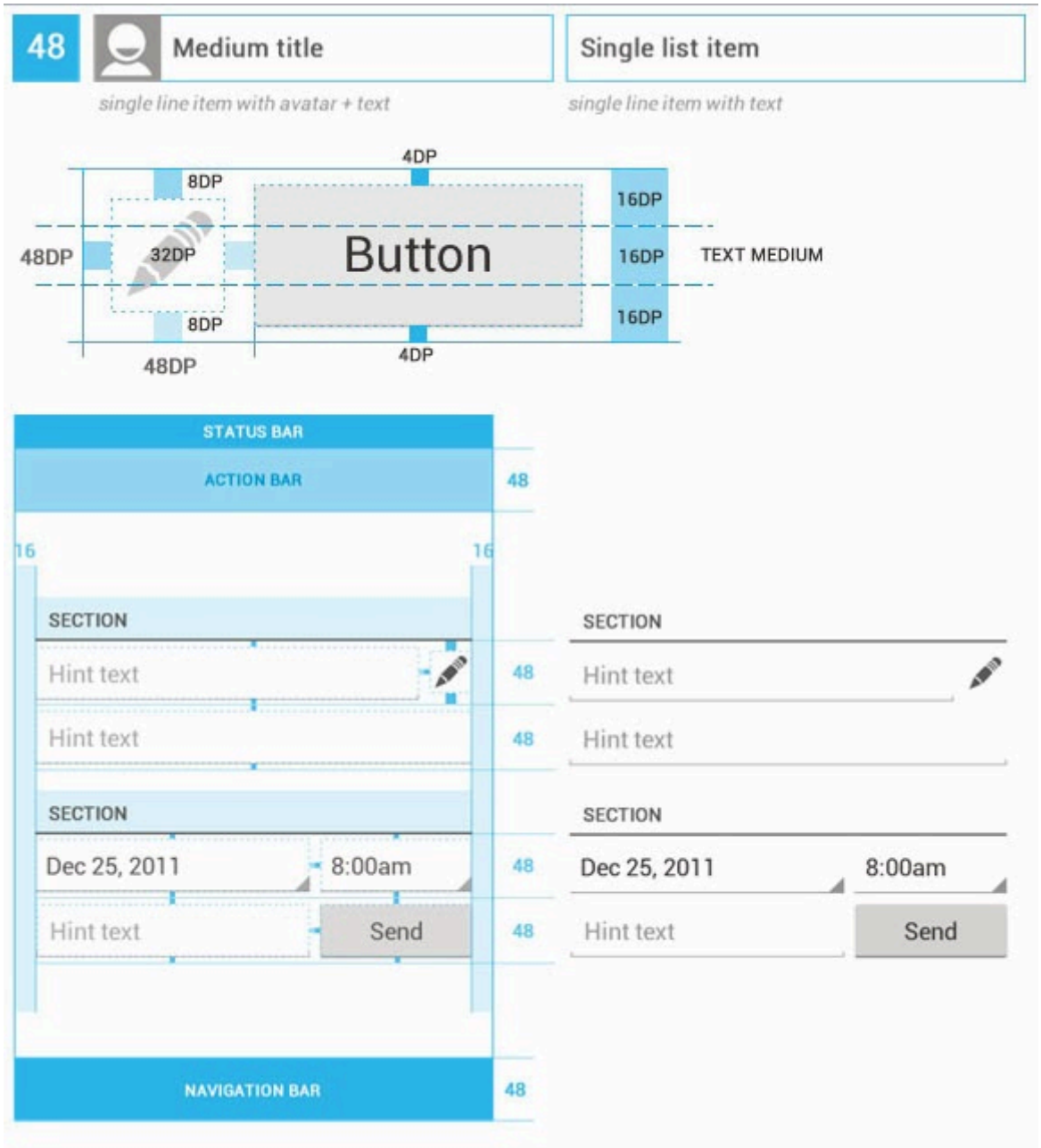
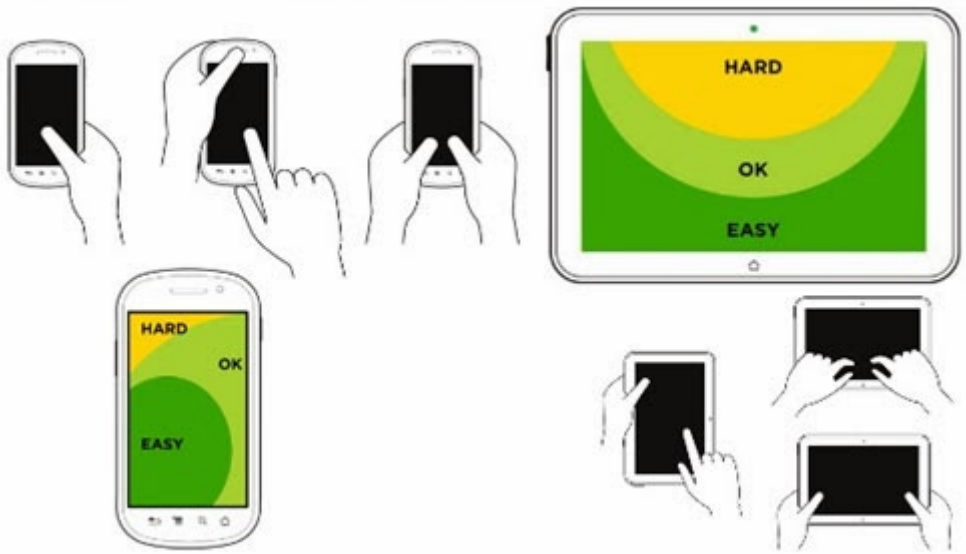


Imagen de http://developer.android.com/design/style/metrics-grids.html

Ten además en cuenta que hay zonas de pantalla más fáciles de pulsar.



Imágenes de http://www.lukew.com/ff/entry.asp?1649

Buena práctica 3: El foco sigue siendo importante en los dispositivos móviles

Sigue siendo importante, tanto que el foco sea visible (no debes ocultarlo en la CSS) como que el orden del foco sea correcto. Muchos usuarios utilizan teclados externos o los usuarios de lector de pantalla deslizan el dedo por la pantalla tabulando de un elemento a otro.

Buena práctica 4: Prueba

Testea tu página, pruébala sin CSS o sin imágenes cargadas, pruébala con un lector de pantalla, prueba con usuarios y si tienes la oportunidad con usuarios con discapacidad. Henny Swan tiene un artículo muy interesante con una lista concreta de comprobaciones recomendadas [SWAN, 2012].

También deberías conocer las Mobile Web Best Practices del W3C, recomendación desde 2008. Hablé de estas y su relación con las WCAG en el artículo "Accesibilidad y usabilidad móvil: web móvil y app nativa", blog Olga Carreras, mayo 2012.

Bibliografía

[AVILA, 2013] "What Does Responsive Web Design Have to do with Accessibility?", Jonathan Avila, julio 2013

[ANDROID] "Metrics and Grids", developer.android.com

[BBC] Mobile Accessibility Guidelines v0.8, BBC

[CARRERAS, 2011] "Ocultar contenido sin comprometer la accesibilidad ni el posicionamiento de la página", Olga Carreras, noviembre 2011

[CARRERAS, 2014] "Screen Resolutions: mobile, Tablet, desktop", Olga Carreras, enero 2014

[CHELARIU, 2013] "Responsive Web Design With Physical Units", Radu Chelariu, marzo 2013

[DEVLIN, 2012] "Responsive HTML5 video", Ian Devlin, agosto 2012

[DESIGNMODO, 2011] "Responsive Web Design: 50 Examples and Best Practices", designmodo.com, octubre 2011

[MEJIA, 2012] "Guía de Responsive Web Design: todo lo que necesita saber sobre Responsive Web Design", Juan Carlos Mejía Llano, enero 2012

[PRIETO, 2014] "Mobile First: Diseñando la Web para el futuro", Ricardo Prieto, enero 2014

[PUKHALSKI, 2014] "Rethinking Responsive SVG" en *Smashing Magazine*. Ilya Pukhalski, marzo 2014

[QUESENBERY, 2010] "Accessibility First—for a Better User Experience for All", Whitney Quesenbery, December 2010

[SOUEIDAN, 2014] "Making SVGs Responsive with CSS", Sara Soueidan, Agosto 2014

[SWAN, 2012] "Mobile accessibility tests", Henny Swan, octubre 2012

[SWAN (b), 2012] "Skip links on mobile and tablets", Henny Swan, abril 2012

[SWAN, 2013] “Introduction to Mobile Accessibility”, Henny Swan, mayo 2013

[UXMATTERS, 2013] “Responsive Web Design and Accessibility”, Uxmatters, abril 2013

[W3C,2008] “Mobile Web Best Practices 1.0”, W3C, 2008

[W3C,2012] Media Queries, W3C, W3C Recommendation 19 June 2012

[W3C_GROUP, 2012] “Main page. Responsive Images Community Group”, W3C

[W3C_GROUP(b), 2012] “Respimg. Responsive Images Community Group”, W3C

[WAHLBIN, 2013] “Responsive Web Design”, Katheleen Wahlbin, noviembre 2013

[WCAG2, 2008] “WCAG 2.0”, W3C, 2008

[WEBAIM, 2012] “Thread: Responsive Web Design and Accessibility?”, Webaim, octubre 2012

[WROBLEWSKI, 2009] “Mobile First”, Luke Wroblewski, noviembre 2009

[WROBLEWSKI, 2010] "Mobile First: One Year Later", Luke Wroblewski, noviembre 2010

[WROBLEWSKI, 2012] "Responsive Navigation: Optimizing for Touch Across Devices", Luke Wroblewski, noviembre 2012

Artículos relacionados

- Claves para la web móvil, diciembre 2012
- Accesibilidad y usabilidad móvil: web móvil y app nativa, mayo 2012